

This is the definitive Project Structure & Solution Architecture for the V1 DTE Platform. As your Principal Software Architect, I recognize that the decisions made in this document will dictate the daily friction, velocity, and compliance of the engineering team for the next decade. A brilliant domain model will decay rapidly if the physical solution structure does not violently defend its boundaries.

This document defines the physical organization of the approved C#/ .NET 8 stack, enforcing the architectural and security mandates previously established.

SECTION 1 — Solution Architecture Philosophy

To guarantee that the platform remains maintainable, auditable, and extensible over a 10-year horizon, the physical solution organization is governed by the following principles:

- **Compiler-Enforced Clean Architecture:** We do not rely on developer discipline to protect the domain. We use the .NET compiler and assembly references (.csproj) to make it physically impossible for the Domain to depend on the Database or Web API.
- **Vertical Cohesion over Horizontal Sprawl:** Inside the architectural layers, code is organized by Bounded Context (e.g., Issuance, Contingency), never by technical type (e.g., no folders named Services or Managers).
- **Zero-Friction Auditing:** The structure must make it immediately obvious to an external auditor where the business rules live versus where the infrastructure logic lives.
- **Pragmatic Assembly Count:** We reject "Assembly Explosion" (creating a separate .csproj for every minor concept) as it destroys IDE performance and developer velocity. We rely on a lean assembly structure combined with strict namespace governance.

SECTION 2 — Repository Strategy

Recommendation: Monorepo

- **Rationale:** The V1 DTE Platform represents a single, cohesive deployment unit (a self-contained binary or Docker container). A monorepo guarantees atomic commits; a change to a DTE schema in the Domain is committed simultaneously with the API contract update and the PostgreSQL migration. There is a single version of the truth at any given Git commit.
- **Tradeoffs:** Monorepos can become large, potentially slowing down CI/CD pipelines.
- **Mitigation:** .NET 8 offers exceptional incremental build caching. We will enforce strict project references so that modifying the Api layer does not trigger recompilation of the Domain layer.
- **Long-Term Implications:** It prevents "integration hell" across micro-repositories, vastly simplifying onboarding for new engineers and ensuring that self-hosted customers always receive a perfectly synchronized set of features.

SECTION 3 — Solution Structure

The solution (.sln) is divided into four strictly controlled layers, representing the Onion/Clean Architecture model.

1. Core Domain Layer

- **Purpose:** The absolute center of the application. Contains the Ubiquitous Language,

Aggregate Roots, Entities, Value Objects, Domain Events, and Domain Exceptions.

- **Ownership:** The business invariants (e.g., the Snapshot Law, Document Totals reconciliation).
- **Forbidden Responsibilities:** Database access, JSON serialization attributes, HTTP requests, Hangfire dependencies, or logging frameworks.

2. Application Layer

- **Purpose:** The orchestration layer. Contains the Use Cases (CQRS Commands and Queries via MediatR), abstract interface definitions for external services (e.g., ITransmissionQueue), and authorization preconditions.
- **Ownership:** Transaction boundaries, audit trail orchestration, and workflow execution.
- **Forbidden Responsibilities:** Direct database queries (must use interfaces), HTTP routing, or handling external API payloads.

3. Infrastructure Layer

- **Purpose:** The integration layer. Contains EF Core DbContexts, Hangfire job implementations, cryptographic key vault wrappers, and external MH API clients.
- **Ownership:** PostgreSQL persistence, Hangfire background processing, file generation (PDF representations), and actual cryptographic hashing logic.
- **Forbidden Responsibilities:** Business logic, state machine rules, or making decisions about *what* to save (it only knows *how* to save).

4. Web API (Presentation) Layer

- **Purpose:** The entry point. Contains ASP.NET Core Controllers, JWT authentication middleware, and global exception handlers.
- **Ownership:** HTTP request/response mapping, API routing, and HTTP status code translation (e.g., mapping a Domain Exception to a 422 Unprocessable Entity).
- **Forbidden Responsibilities:** Business logic, direct database access, or orchestrating background jobs directly.

SECTION 4 — Assembly Boundaries

To prevent architectural erosion, the solution is divided into four core assemblies.

Dependencies flow strictly inwards.

Assembly (.csproj)	Responsibilities	Allowed Dependencies	Forbidden Dependencies
DTE.Domain	Enterprise business rules, Aggregates, Value Objects.	None (No external Nuget packages except fundamental math/types).	DTE.Application, DTE.Infrastructure, DTE.Api, EF Core, Hangfire.

DTE.Application	CQRS Handlers, interfaces, orchestration.	DTE.Domain, MediatR.	DTE.Infrastructure, DTE.Api, EF Core, Hangfire.
DTE.Infrastructure	EF Core, PostgreSQL, Hangfire, external API implementations.	DTE.Domain, DTE.Application, EF Core, Hangfire, AWS/Azure SDKs.	DTE.Api.
DTE.Api	ASP.NET Core, Controllers, Middleware.	DTE.Application, DTE.Infrastructure (strictly for Dependency Injection setup).	None.

Dependency Rule: The Api assembly references Infrastructure solely to wire up Dependency Injection in Program.cs. Controllers must only inject interfaces defined in Application.

SECTION 5 — Module Placement Rules

We will not create 40 separate assemblies. Instead, the 10 approved modules are implemented as **Feature Namespaces** that slice vertically through the assemblies.

- **Issuance, Invalidation, Contingency:** * *Domain:* DTE.Domain.Issuance, DTE.Domain.Invalidation, DTE.Domain.Contingency. Contains the aggregates (FiscalDocument, InvalidationEvent, ContingencyEpisode).
 - *Application:* DTE.Application.Issuance.Commands, DTE.Application.Issuance.Queries. Contains the MediatR handlers.
- **Transmission:**
 - *Domain:* DTE.Domain.Transmission (contains TransmissionRecord).
 - *Infrastructure:* DTE.Infrastructure.Transmission.MHApiClient (implements the actual HTTP calls to the authority).
- **Audit & Evidence:**
 - *Domain:* DTE.Domain.Audit, DTE.Domain.Evidence (contains AuditEntry, EvidenceBundle).
 - *Application:* Cross-cutting pipelines (MediatR behaviors) that intercept commands to orchestrate audit generation and evidence hashing.
 - *Infrastructure:* DTE.Infrastructure.Evidence.Storage (handles the PostgreSQL BYTEA persistence).
- **Catalog, Identity, Configuration:**
 - Placed in their respective namespaces (DTE.Domain.Catalog, etc.). Identity integration interfaces live in Application, while the JWT generation and password hashing live in Infrastructure.

- **Regulatory Bundle:**
 - *Application:* Command handlers to process incoming bundles.
 - *Infrastructure:* Cryptographic signature verification implementations to ensure the vendor bundle is authentic.

SECTION 6 — Dependency Enforcement Model

To ensure junior engineers or rushed features do not compromise the 10-year lifespan, boundaries are enforced automatically:

1. **Compile-Time Enforcement:** The .csproj references physically prevent the Domain from importing Infrastructure. If a developer attempts to add Microsoft.EntityFrameworkCore to the Domain, the PR will immediately flag the anomalous package reference.
2. **Architectural Unit Testing (NetArchTest):** We will utilize the NetArchTest.Rules library inside our unit test suite.
 - *Rule:*
Types().That().ResideInNamespace("DTE.Application").Should().NotHaveDependencyOn("DTE.Infrastructure")
 - *Rule:*
Classes().That().Inherit(typeof(IQueryHandler)).Should().HaveNameEndingWith("Handler")
 - *Enforcement:* If an architectural test fails, the CI pipeline fails. The build is rejected.

SECTION 7 — Background Processing Structure

Hangfire is the approved engine, but it must not pollute our business logic.

- **Hangfire Ownership:** Hangfire configuration, dashboards, and job enqueueing implementations live exclusively in DTE.Infrastructure.BackgroundProcessing.
- **Job Organization:** Hangfire "Jobs" are extremely thin wrappers. A Hangfire job class takes a serialized JSON command, deserializes it, and sends it to MediatR: `_mediator.Send(command)`. Hangfire contains zero business logic.
- **Retry Ownership:** Hangfire controls the physical network retry loops (e.g., retrying an MH transmission on a 502 Bad Gateway).
- **Scheduling Ownership:** The Application layer defines an interface `IBackgroundJobClient`. The MediatR handler calls this interface. The Infrastructure layer implements this interface using Hangfire, cleanly decoupling the application from the specific background tool.

SECTION 8 — Integration Structure

V1 focuses on UI, but V1.5 ERP integration must be anticipated seamlessly.

- **ERP Integrations:** Managed via the DTE.Api layer. New endpoints (`/api/v1/integration/documents`) will map external ERP JSON structures to internal CQRS Commands (`CreateDraftCommand`). The core Issuance module remains oblivious to whether the command originated from the UI or an ERP.
- **Import/Export Services:** * *Ownership:* DTE.Application.Reporting.
 - *Boundaries:* Exporting the "Libro de Ventas" relies on CQRS Queries reading directly from the database snapshots, entirely bypassing the heavy Domain aggregates to

maximize performance.

- **Synchronization Services:** Batch updates to Counterparties or Items from an ERP invoke the `UpdateCounterpartyCommand` iteratively. The Snapshot Law guarantees that historical documents remain untouched.

SECTION 9 — Shared Components Policy

The creation of a "Shared Junk Drawer" (Common, Utils, Helpers) is **strictly forbidden**. These namespaces become unmaintainable dumping grounds.

- **What May Be Shared:**
 - `DTE.Domain.SeedWork`: Abstract base classes necessary for DDD (Entity, AggregateRoot, ValueObject, DomainEvent).
 - `DTE.Domain.Exceptions`: Base domain exception types.
 - `DTE.Application.Behaviors`: MediatR pipeline behaviors for logging, auditing, and validation.
- **What Must Never Be Shared:**
 - String manipulation utilities (push them close to the feature that uses them).
 - "Generic" database repositories (we use specific Use Cases, not generic data access layers).

SECTION 10 — Testing Structure

Testing projects mirror the assembly structure directly:

- **DTE.Domain.Tests (Unit Tests):** Tests the state machine, totals reconciliation, and invariant enforcement. Fast, memory-only, no dependencies.
- **DTE.Application.Tests (Unit/Integration Tests):** Tests that CQRS handlers orchestrate logic correctly. Mocks external interfaces (like the MH API client) but uses real domain logic.
- **DTE.Infrastructure.Tests (Integration Tests):** Uses **Testcontainers** (Docker-in-Docker) to spin up a real PostgreSQL database. Tests EF Core mappings, Hangfire job enqueueing, and PostgreSQL constraint enforcement.
- **DTE.Api.Tests (End-to-End Tests):** Uses `WebApplicationFactory` to boot the entire API in-memory (connected to a Testcontainers database). Tests HTTP routing, authentication, and end-to-end JSON serialization.

SECTION 11 — Evolution Strategy

This architecture is designed to absorb the future without requiring structural restructuring:

- **Additional DTE Types (e.g., 11, 14):** Handled purely as data. New JSON schema configurations are loaded into the Document Type Registry. The Issuance domain logic evaluates against the registry dynamically; no code changes in `DTE.Domain.Issuance` are required.
- **Approval Workflows:** Handled via the Application layer. We introduce a MediatR pipeline behavior that intercepts `ValidateDocumentCommand`. If a configuration flag requires approval, the pipeline halts the execution and transitions the command to a new `DTE.Domain.Approvals` bounded context.
- **Inbound DTE Reception:** Handled as a net-new feature namespace

(DTE.Domain.Inbound, DTE.Application.Inbound). It will have its own aggregates (e.g., SupplierDocument) ensuring accounts payable logic never infects accounts receivable logic.

- **Multi-Emisor Support:** Handled via EF Core Global Query Filters in the Infrastructure layer. A TenantId (IssuerId) is extracted from the JWT token and automatically appended to all database queries, requiring almost zero changes to the Application or Domain logic.

SECTION 12 — Architectural Risk Review

To ensure robustness, we must actively attack this proposed structure.

Risk 1: CQRS Leakage (The "Fat Query" Problem)

- *Failure Mode:* Developers start using EF Core to load heavy Domain Aggregates just to display a list on a UI screen, causing massive memory spikes and N+1 query performance issues.
- *Mitigation:* NetArchTest rules will enforce that IQueryHandler classes return primitive DTOs or ViewModels, never AggregateRoot classes. Queries must use raw SQL (Dapper) or heavily projected EF Core .Select() statements reading from the JSONB snapshots.

Risk 2: The Monolith Becomes a Big Ball of Mud

- *Failure Mode:* Over 5 years, the vertical feature namespaces lose discipline. The Issuance command handlers start directly referencing Counterparty database tables.
- *Mitigation:* Strict Code Review guidelines. Cross-aggregate updates must be handled by Domain Events. If Issuance needs data from Catalog, it must query it via a defined interface, never directly coupling to the other module's EF Core DbSet.

Risk 3: MediatR Hidden Dependencies (Event Cascades)

- *Failure Mode:* A command triggers a Domain Event, which triggers a handler, which triggers another event, creating an untraceable, synchronous loop that times out the HTTP request.
- *Mitigation:* Synchronous Domain Events (INotificationHandler) are strictly limited to in-transaction boundary logic (e.g., generating the Audit Entry). Cross-module events (e.g., notifying the Catalog that an item was used) must be dispatched asynchronously via Hangfire to immediately release the HTTP thread.